# Data Science

## K-Nearest Neighbors

GENERAL ASSEMBLY

Machine Learning Fundamentals

# Lesson roadmap

- Welcome + Warm-up
- Introduction to k-NN
- Data prep for k-NN
- k-NN in scikit-learn
- Bring it Home

# Learning objectives

1. Understand how classification works.
2. Describe the k-NN algorithm and the intuition behind it.
3. Prepare data for k-NN algorithm using standardization techniques.
4. Fit, evaluate, and generate predictions from a k-NN model in `scikit-learn`.

# What is Classification?

## An overview of the key concepts

# Two Kinds of Supervised Learning

Supervised learning models fall into two different buckets:

**Classification**

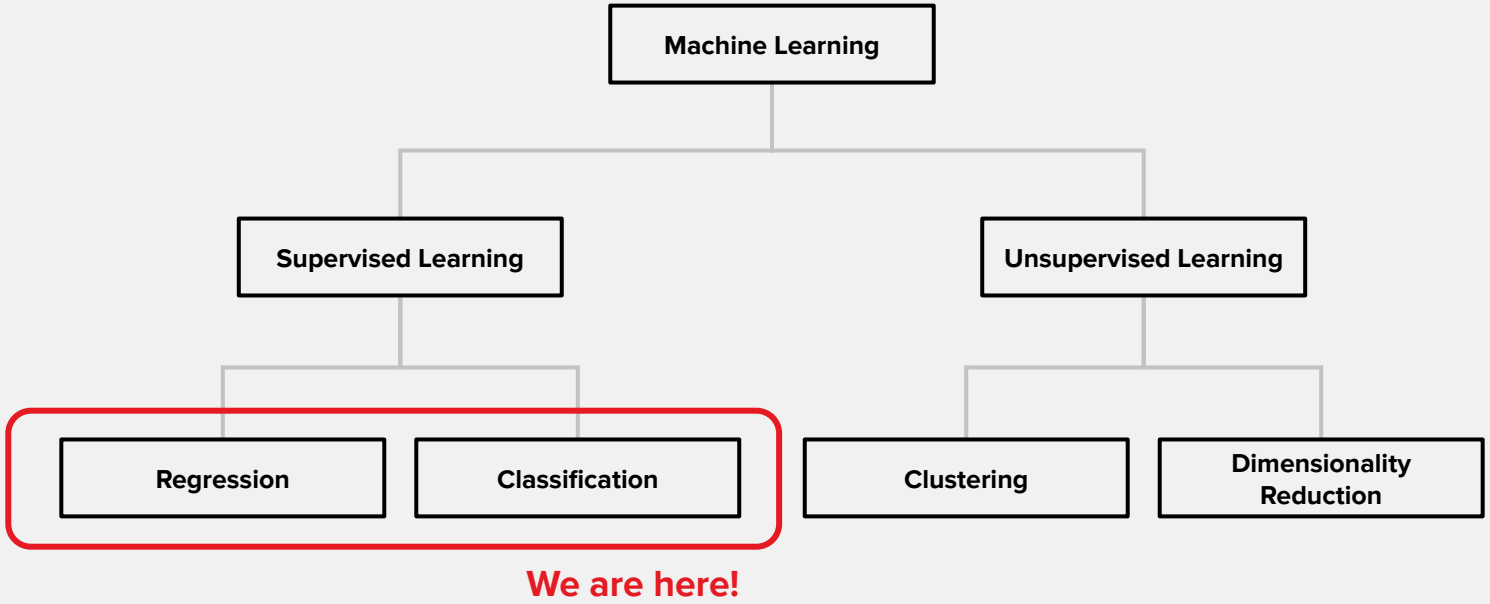
This is when our *y*-variable is a category. A 0/1 yes/no classification is often called **binary classification**. Otherwise, **multiclass classification**.

**Regression**

This is when our *y*-variable is numeric.

 © 2025 General Assembly

# Where are we now?



# Classification

Classification is the branch of supervised learning in which we are concerned with predicting a **categorical response**, oftentimes a binary (yes/no) response.

For example:

**“Given this person’s demographics, will they vote for candidate A or B?”**

Source

 © 2025 General Assembly

# Classification

Classification is the branch of supervised learning in which we are concerned with predicting a **categorical response**, oftentimes a binary (yes/no) response.

For example:

**“Given this person’s financial status, will they default on their loan?”**



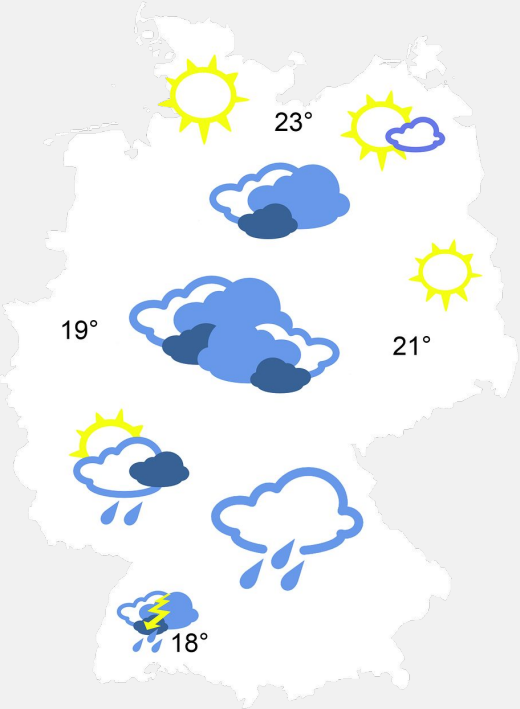
Source

# Classification

Classification is the branch of supervised learning in which we are concerned with predicting a **categorical response**, oftentimes a binary (yes/no) response.

For example:

**“Will it rain tomorrow?”**




Source

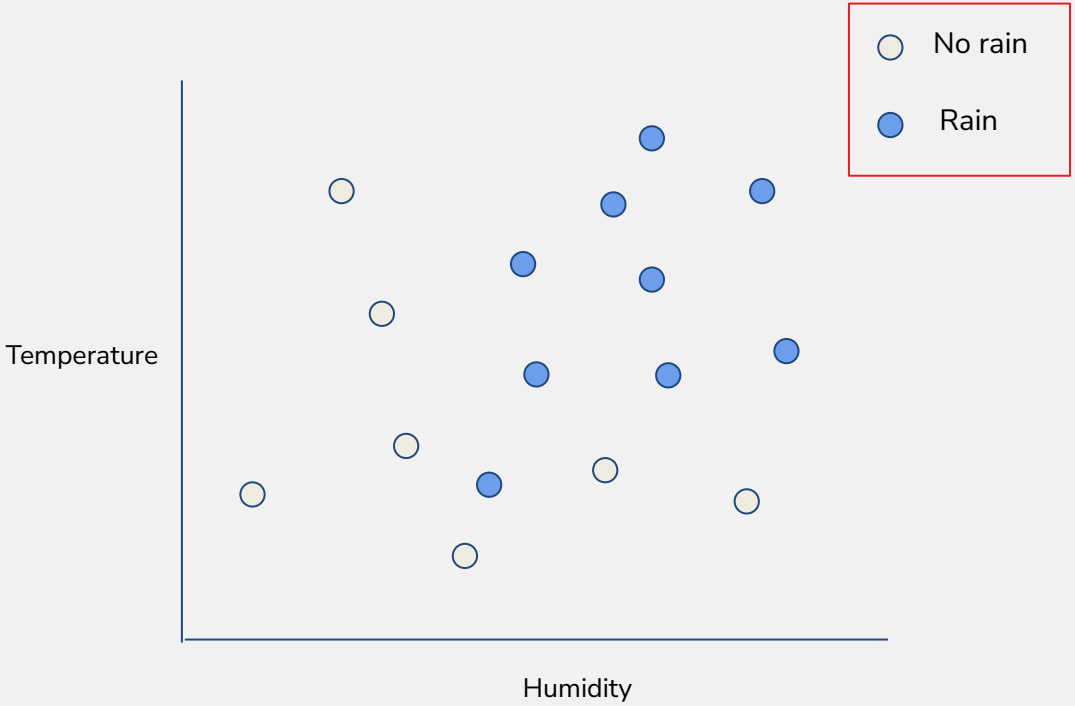
# Introduction to K-Nearest Neighbors (k-NN)

# A classification problem

Imagine you have some data.

Each point consists of features (temperature, humidity) and a response (rain/no rain).

The data is represented on the chart to the right.

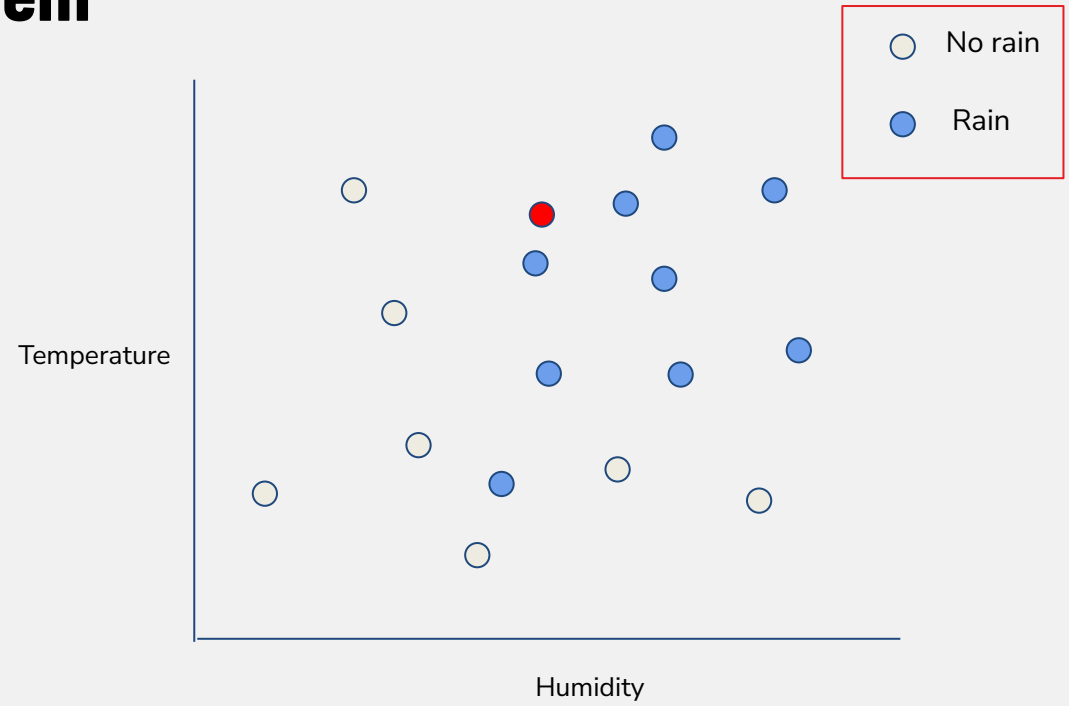


# A classification problem

A new point appears!

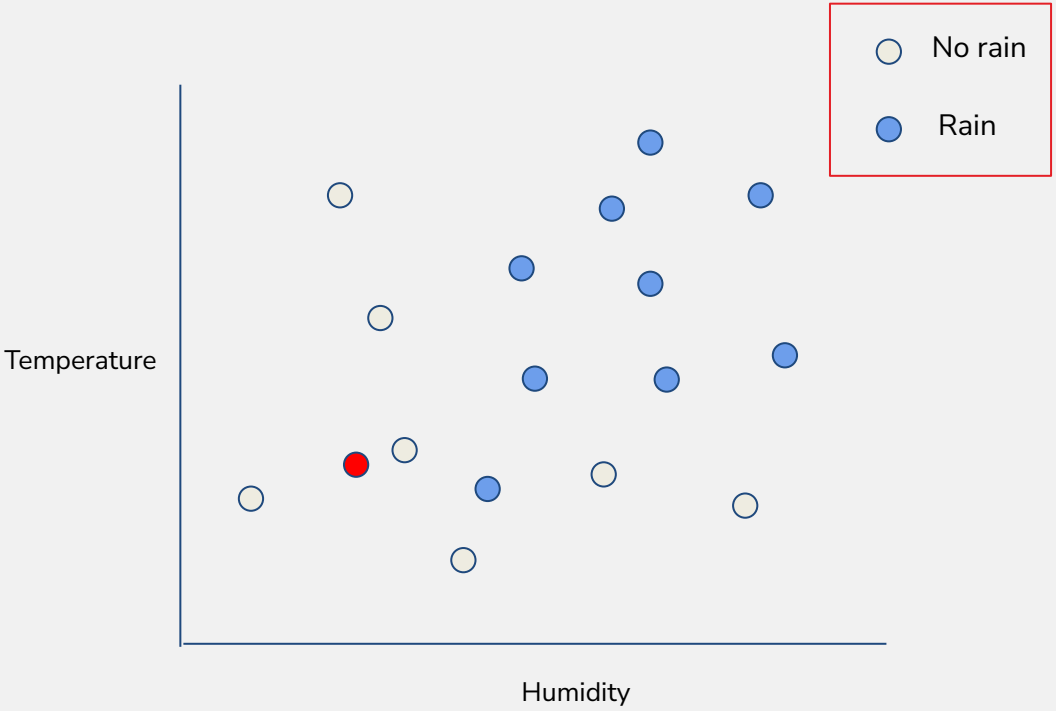
We know the value of its two features (humidity, temperature) but we **don't know its label** (i.e. whether that point represents a day when it rained or not).

How will you decide which label to give this point (rain/no rain) using only the information on this chart?



# Which label?

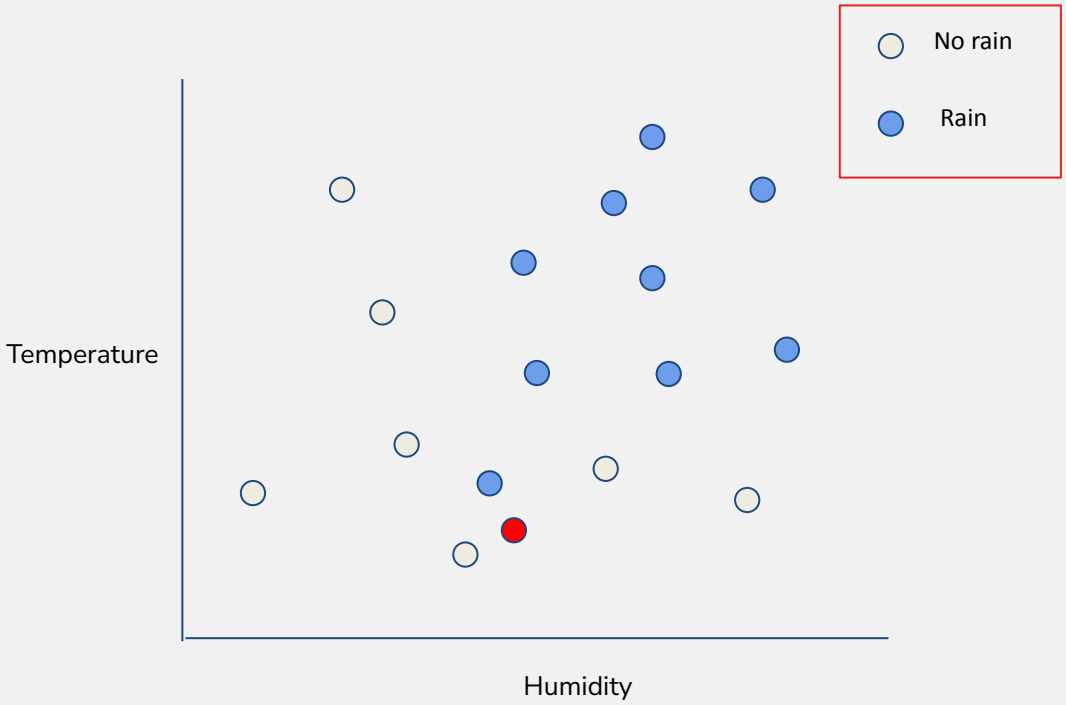
How will you decide which label to give this point (rain / no rain) using only the information on this chart?



# ...and now?

Now our new point is **equidistant** from a rainy day and a dry day.

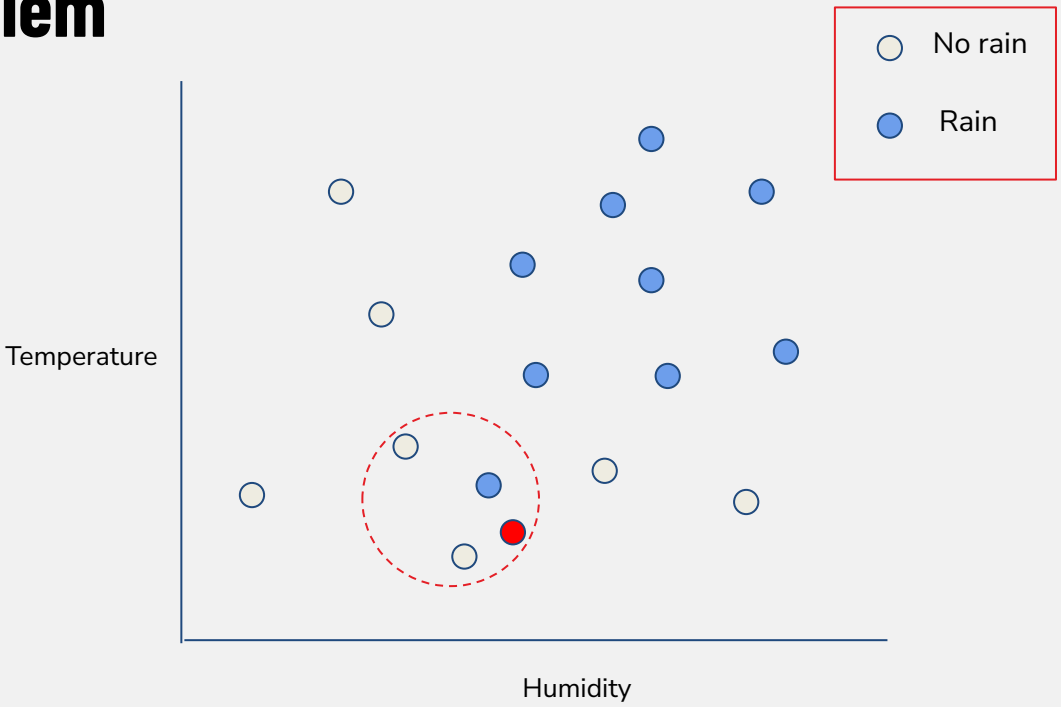
How should we make our classification now?



# A classification problem

Instead of looking at the **one** closest point, we might look at the **three** closest points to our new point.

This would give us a more accurate classification.

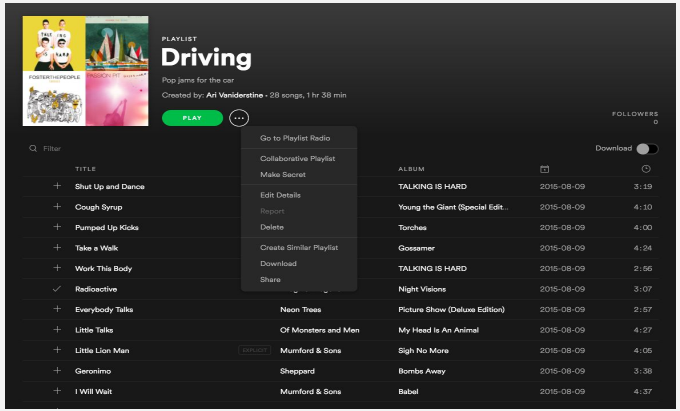


# What Is K-Nearest Neighbors (KNN)?

Suppose we want to determine your favorite type of music. How might we determine this without directly asking you?

Generally, friends share **similar traits and interests** (e.g. music, sports teams, hobbies, etc). We might ask your five closest friends what their favorite type of music is and take the majority vote.

**This is the idea behind KNN:** we look for points similar to (or close to) our new observation and identify shared traits. We can use this information to make an educated guess about the class of our new data point.




Source: data-to-viz.com

# What is k-NN?

k-Nearest Neighbors is a **non-parametric** algorithm. A non-parametric algorithm makes **no assumptions about the distribution your data**. k-NN can be used with both classification and regression problems.

It follows the "**birds of feather flock together**" logic where it assumes **data points that are near each other are similar**.

 © 2025 General Assembly

# What Is K-Nearest Neighbors (KNN)?

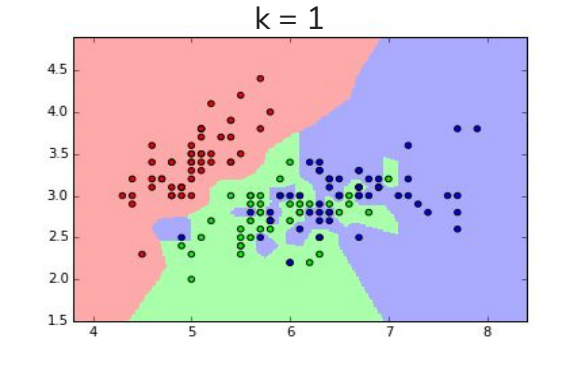
**K-nearest neighbors (KNN)** is a classification algorithm that classifies a data point using information about its **k**-nearest neighbors.

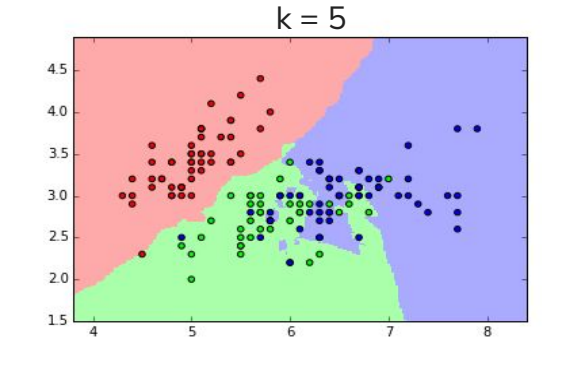
## How the KNN algorithm works:

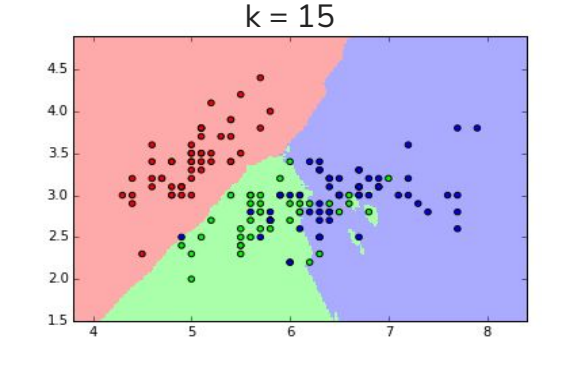
- For a given point, calculate the distance to all other points.
- Given those distances, pick the *k* closest points.
- The predicted class label is the majority class among the *k*-nearest neighbors surrounding the point.

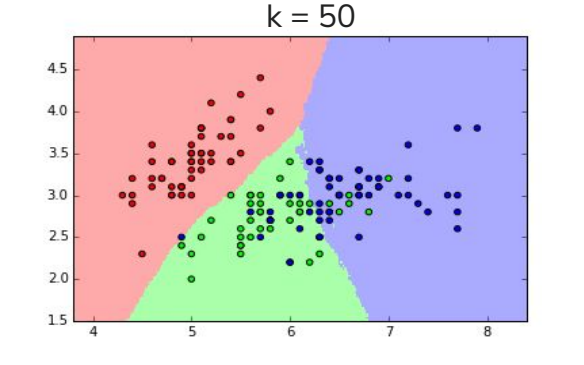
 © 2025 General Assembly

# What Does KNN Look Like?



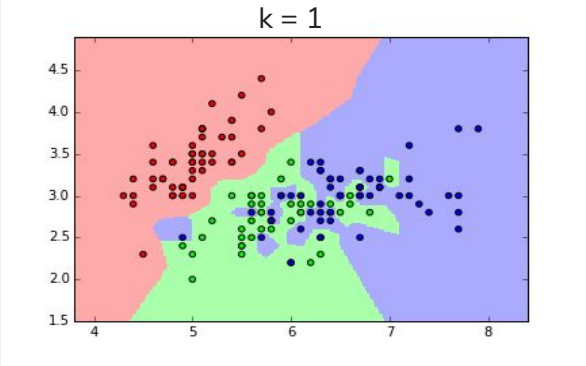



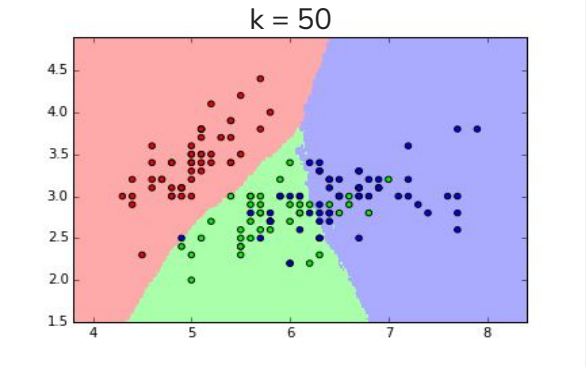




The dots represent actual data (that the model is trained on). The colored space shows the **decision boundaries** for any new data being evaluated (e.g. any new data point in the red area would be allocated as category red, etc).

These are known as Voronoi diagrams.

# How do we interpret K?




We can see that, as K increases, the decision boundaries become more distinct. However, you can also see that the spaces are not perfectly pure when it comes to the known elements within them.

**How are outliers affected by K?** As K increases, outliers are "smoothed out". Look at the plots and notice how outliers strongly affect the prediction space when K=1. When K=50, outliers no longer affect region boundaries. This is a classic **bias-variance tradeoff**: with increasing K, the variance decreases but the bias increases.

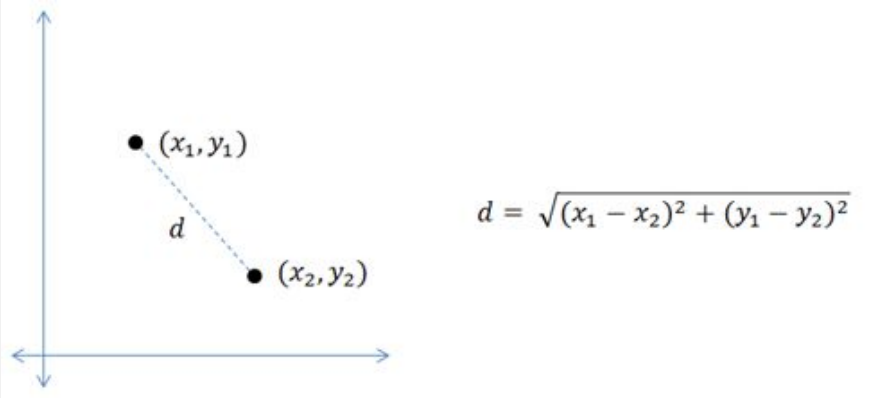
 © 2025 General Assembly

# How do we define ‘nearest’?

We measure the distance between data points using the **Euclidean** or **straight line** distance.

You might remember calculating the Euclidean distance as part of Pythagoras’ theorem at school. This is a **2D** example.

We can also calculate the Euclidean distance between points with more than two dimensions.




 © 2025 General Assembly

# There are two other common distance measures used

**Manhattan distance** (i.e. taxi distance)

**Minkowski** is similar to the other two

Euclidean $$\sqrt{\sum_{i=1}^{k}(x_i - y_i)^2}$$

Manhattan $$\sum_{i=1}^{k}|x_i - y_i|$$

Minkowski $$\left(\sum_{i=1}^{k}(|x_i - y_i|)^q\right)^{1/q}$$

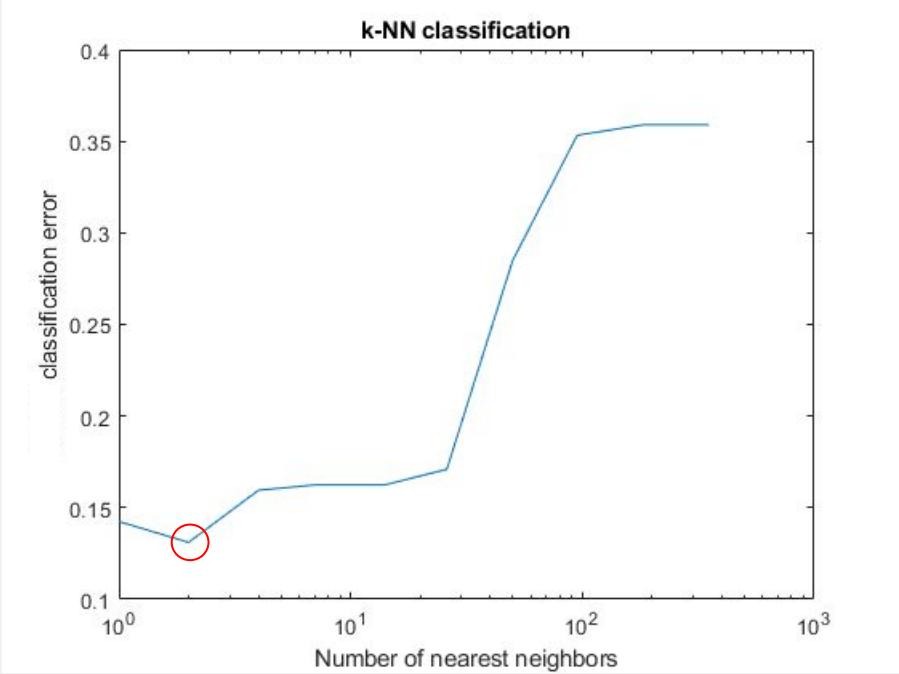
 © 2025 General Assembly

# How do we choose k?

Picking **k** is a manual choice that we need to make each time we train a KNN model.

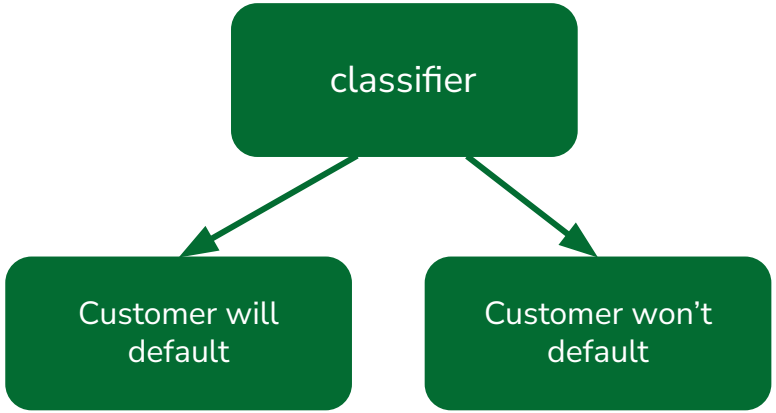
We'll typically train and test KNN models with different values of k, and pick the value that gives the lowest error on our **testing set**.

Error here is 1 minus the accuracy.




 © 2025 General Assembly

# Measuring how good a model is




Here's a trained classifier that makes decisions about whether people will default on loans or not.

How can we estimate the accuracy of this model on data it hasn't seen before?

# How Good is Your Model?

For regression models, we use **mean squared error** or **root mean squared error** to measure model performance.

But that won't work for classifiers, because the output isn't a number: a classifier can either be **wrong** or **right**.

**A simple way of measuring the model's accuracy on the testing dataset is to work out the % of times the classifier makes the correct prediction. This is the number of times each category (wrong and right) are correctly predicted.**

**This is the accuracy score.**

 © 2025 General Assembly

# What does accuracy mean?

...but an overall accuracy score can be misleading.

For example: a test for a very rare but very serious disease is 99% accurate.

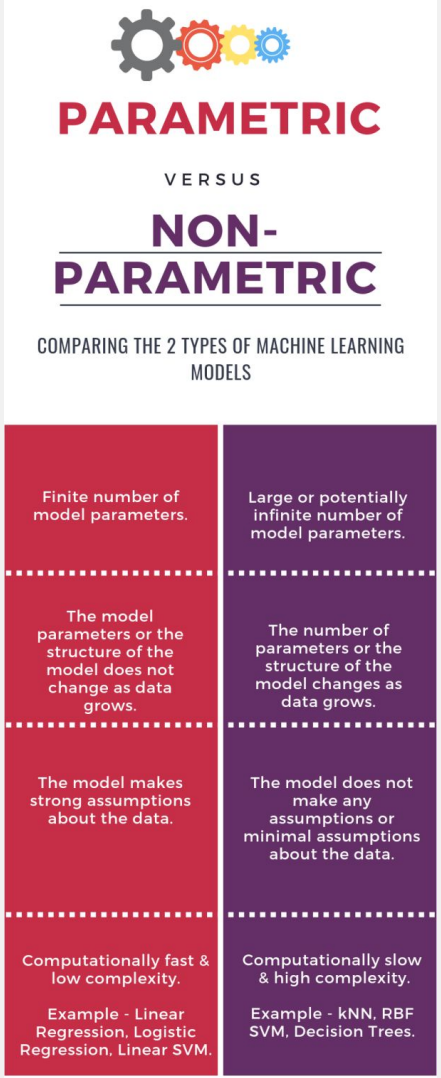
Do you think this is a good accuracy figure? What else would you want to know about the test?

We need to know the **baseline** (or Null), which for classification is the most frequently occurring class in the training dataset.

 © 2025 General Assembly

# knn is a non-parametric model

The model is not represented as an equation with parameters (e.g. the values in linear regression).




 © 2025 General Assembly

# Recap of fitting steps

## Steps of k-NN

1. Calculate the distance between all of the points (red lines) this could be calculated in several ways. Euclidean (or straight line) distance is the most common
2. Rank each point by distance (smallest to largest)
3. Take a vote (for classification) or average  (for regression) of the k nearest points
4. Count the votes and the majority becomes the prediction

You can decide the value of k, using the value that improves the quality of your predictions without overfitting.

 © 2025 General Assembly

# Data Prep for k-NN

## How to prepare your data for k-NN

# Scaling data

Because KNN is calculating the distance between neighbors, it's **highly sensitive to the magnitude of your features**. For example, if we were using k-NN on a housing dataset, a feature like **square footage** (measured in **thousands** of feet^2) can really affect the distance.

Thus, in order for k-NN to work properly, it's **important to scale or normalize our data**. We can do that with built in tools in `scikit-learn`.

# Other considerations

k-NN struggles with large datasets and can become very slow with large data.

- Avoid too many rows or columns.
- Clean and standardize your data! k-NN is very sensitive to extreme values and irrelevant values.
- Consider dropping categorical columns unless they are critical to the analysis because dummy coding will potentially add a lot of extra columns.
- Experiment with the correct k value. It defaults to 5.

k-NN is widely used for image and video processing and other tasks where similar items (like pixels in an image) need to identified.

© 2025 General Assembly

# k-NN

Let's see it for ourselves!

- ☐ Open **02-knn.ipynb**

# Bring it Home

## Summary

# Key takeaways

k-NN

## 1 Strengths

Non-parametric so it has few data assumptions.

Can be used with both classification (vote) and regression (average) problems.

## 2 Weaknesses

Can be computationally and memory expensive.

Sensitive to noisy, imbalanced, and

irrelevant data.

Works poorly if you have many predictors.

Choosing k can be tricky!

## 3 Metrics

Regression: Error (MSE, RMSE, etc.)

Classification: Accuracy, ROC AUC, precision, recall, F1-Score.

## 4 When use

Predicting continuous or categorical outcomes.

With a small number of predictors and relatively small number of rows.

GENERAL ASSEMBLY